

CloneCalls

Initial Implementation & Secure Coding Report

Deliverable 2

1. Project Overview

CloneCalls is a web application that allows users to create an AI-powered voice clone that answers calls on their behalf. A user briefs their clone with daily context — schedule, availability, messages to relay — then an authorised caller can query the clone via text or voice and receive a spoken AI-generated response in real time.

Because the system handles personal context, voice recordings, and authenticated user sessions, security was treated as a first-class design requirement throughout the implementation — not added retrospectively.

1.1 System Architecture

Component	Technology	Role
Backend API	Python FastAPI	REST API serving all business logic and security controls
Authentication	Supabase Auth	JWT issuance, password hashing, user management
Vector Memory	Pinecone DB	Stores and retrieves per-user briefing embeddings
Language Model	OpenAI GPT-4o-mini	Generates clone responses; Whisper transcribes audio
Voice Synthesis	ElevenLabs TTS	Converts clone text responses to spoken audio
Frontend	HTML / Vanilla JS	Single-page application served via Live Server

1.2 Security Design Philosophy

Four principles drove security decisions across the codebase:

- Server-side identity enforcement — user identity always derived from the validated JWT, never from request body fields
- Multi-layer access control — data access requires matching three simultaneous filters, not just authentication
- Fail-fast secrets validation — server refuses to start if any credential is missing from .env
- Defence in depth — independent security controls at frontend, API boundary, application, and data layers

2. Threat Model

2.1 System Assets

Asset	Description	Risk if Compromised
User credentials	Email addresses and hashed passwords in Supabase	Account takeover
JWT access tokens	Bearer tokens authenticating all protected API calls	Full user impersonation
Briefing memories	Personal daily context stored as vectors in Pinecone	Privacy violation, data leak
Voice audio	Raw .webm recordings uploaded by callers	Sensitive audio persists on server
API keys	Supabase, OpenAI, Pinecone, ElevenLabs credentials	Unauthorised API usage, billing fraud

2.2 STRIDE Threat Analysis

Threat	Category	Attack Scenario	Control Applied
Token forgery	Spoofing	Attacker crafts fake JWT to impersonate user	Server-side Supabase token validation on every request
Cross-user data access	Tampering	User changes target_email to access another user's briefings	Triple-filter Pinecone queries (owner + caller + date)
PII leakage	Info Disclosure	User includes SSN or credit card in briefing text	Regex PII scrubbing before any database write
Prompt injection	Tampering	Caller manipulates AI to reveal out-of-scope data	Constrained system prompt with hard security rule

Memory exhaustion DoS	Denial of Service	Attacker uploads 500 MB audio to exhaust server memory	10 MB file size cap enforced before memory allocation
API enumeration	Info Disclosure	Attacker browses Swagger UI to map all endpoints	docs/redoc/openapi.json all disabled by default
Credential brute force	Elev. of Privilege	Attacker tries thousands of passwords against /login	Generic error messages; Supabase account lockout
Clickjacking	Tampering	App embedded in an iframe to hijack user clicks	X-Frame-Options: DENY + CSP frame-ancestors 'none'
Key exposure in logs	Info Disclosure	API keys appear in server access logs via URL params	All keys passed in HTTP headers only
Hung connection DoS	Denial of Service	Slow ElevenLabs response permanently stalls a thread	30-second timeout on all outbound API calls

3. Secure Coding Implementation

The following sections document each security control implemented in the source code. All controls were written as part of the initial implementation and map directly to threats identified in the STRIDE model above.

3.1 Authentication — JWT Bearer Tokens via Supabase

File: security.py

All protected endpoints use FastAPI's HTTPBearer dependency. On every request the token is validated server-side against Supabase — never decoded locally and trusted blindly. The function returns the verified email from Supabase, not any client-supplied identity.

```
def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security_scheme)):
    try:
        user_response = supabase.auth.get_user(credentials.credentials)
        if not user_response.user:
            raise HTTPException(status_code=401, detail='Unauthorized')
```

```

    return user_response.user.email # verified by Supabase, not client-supplied
except Exception:
    raise HTTPException(status_code=401, detail='Invalid or expired token')

```

Password constraints are enforced at the schema layer — minimum 8 characters, maximum 50 — preventing trivial passwords from being accepted.

Threat Mitigated	How
Token forgery	Token validated against Supabase live on every single request
Session hijacking	Tokens expire; no persistent session state on the server
Unauthenticated access	HTTPBearer dependency blocks any request with no or malformed token

3.2 Authorisation — Triple-Filter Data Isolation

File: routes.py

Every Pinecone query enforces three simultaneous filters. All three must match for any data to be returned. The critical property is that `current_email` comes from the validated Supabase token — not from any user-supplied value in the request body.

```

filter={
    'owner':      target_email.lower(), # whose memories to retrieve
    'allowed_caller': current_email.lower(), # from token — never from request body
    'date':      current_date          # today only — stale data inaccessible
}

```

Filter	What it enforces
owner	Only the target user's memories are returned — not all users in the system
allowed_caller	Only callers explicitly whitelisted by the owner can retrieve the data
date	Only today's briefings are visible — prevents access to historical data

3.3 Input Validation — Pydantic Schemas and Query Patterns

File: schemas.py, routes.py

All inbound request bodies are validated by Pydantic models before any handler logic executes. Query parameters are validated using `Query(pattern=...)` so invalid email addresses are rejected at the API boundary.

```
class UserSignup(BaseModel):
    email: EmailStr
    password: str = Field(..., min_length=8, max_length=50)
    full_name: str = Field(..., min_length=2, max_length=100)

class DailyBriefing(BaseModel):
    text: str = Field(..., min_length=5, max_length=1000)
    allowed_caller: EmailStr
```

Field	Type	Constraint
email	EmailStr	Valid RFC email format
password	str	min 8, max 50 chars
full_name	str	min 2, max 100 chars
text (briefing)	str	min 5, max 1000 chars
allowed_caller	EmailStr	Valid RFC email format
target_email (query)	str	Regex pattern via <code>Query(pattern=...)</code>

3.4 File Upload Security — Size Cap and Guaranteed Cleanup

File: `routes.py`

Audio uploads enforce a 10 MB hard limit before the file is written to disk. An empty file is rejected immediately. The temporary file is deleted in a finally block cleanup is guaranteed even if transcription throws an exception.

```
MAX_AUDIO_BYTES = 10 * 1024 * 1024
audio_bytes = await audio_file.read(MAX_AUDIO_BYTES + 1)
if len(audio_bytes) == 0:
    raise HTTPException(status_code=422, detail='Audio file is empty.')
if len(audio_bytes) > MAX_AUDIO_BYTES:
    raise HTTPException(status_code=413, detail='Audio file too large (max 10 MB).')
try:
    # transcription logic
finally:
    os.remove(temp_audio_path) # always executes, even on exception
```

3.5 PII Scrubbing — Regex Redaction Before Storage

File: `security.py`

Briefing text is scrubbed of sensitive personal data before embedding and storing in Pinecone. The raw text never reaches the vector database.

```
def scrub_pii(text: str) -> str:
    text = re.sub(r'\b(?:\d{13,16})\b', '[REDACTED CREDIT CARD]', text)
    text = re.sub(r'\b\d{3}[-.\s]?d{3}[-.\s]?d{4}\b', '[REDACTED PHONE]', text)
    text = re.sub(r'\b\d{3}-\d{2}-\d{4}\b', '[REDACTED SSN]', text)
    return text
```

3.6 Prompt Injection Defence — Constrained System Prompt

File: routes.py

Both call simulation endpoints include a hard constraint in the AI system prompt that restricts the model to only the briefed memory context. Context is physically scoped to top_k=3 Pinecone results filtered by all three access control fields.

```
CRITICAL SECURITY RULE: You may ONLY use the information provided
in the 'Memories' section below. If the answer is not in the memories,
politely tell the caller that the owner hasn't briefed you on that topic.
```

3.7 CORS Policy — Strict Origin Allowlist

File: main.py

The CORS middleware uses an explicit origin allowlist from .env never a wildcard. Methods restricted to GET and POST only. Headers limited to Authorization and Content-Type only.

```
app.add_middleware(CORSMiddleware,
    allow_origins=ALLOWED_ORIGINS, # from .env — no wildcard
    allow_credentials=True,
    allow_methods=['POST', 'GET'],
    allow_headers=['Authorization', 'Content-Type'],
)
```

3.8 Security Response Headers — Middleware

File: main.py

Header	Value	Attack Prevented
X-Content-Type-Options	nosniff	MIME-type sniffing attacks
X-Frame-Options	DENY	Clickjacking via iframe embedding
Content-Security-Policy	default-src 'none'; frame-ancestors 'none'	Script injection, framing

Cache-Control	no-store, no-cache, must-revalidate	Sensitive data cached in browser/proxy
Referrer-Policy	no-referrer	URL leakage to external services
Permissions-Policy	geolocation=(), camera=()	Unauthorised hardware access requests

3.9 Secrets Management — Fail-Fast Validation

File: config.py

All credentials loaded from .env via python-dotenv. A startup check crashes the server immediately if any key is absent, preventing silent partial-configuration deployments. .env is excluded from the Git repository via .gitignore.

```
if not all([SUPABASE_URL, SUPABASE_KEY, OPENAI_API_KEY,
           PINECONE_API_KEY, PINECONE_INDEX_NAME,
           ELEVENLABS_API_KEY, ELEVENLABS_VOICE_ID]):
    raise RuntimeError('CRITICAL: Missing API keys in .env file')
```

3.10 Error Handling — No Information Leakage

File: routes.py, main.py

- Login always returns 'Invalid email or password' — does not distinguish between wrong email and wrong password, preventing email enumeration
- Signup returns a generic message — does not confirm whether an email address already exists
- ElevenLabs failures return HTTP 502 'Voice synthesis failed.' — raw third-party API response body never forwarded to client
- API documentation disabled by default: docs_url=None, redoc_url=None, openapi_url=None

3.11 External API Hardening

File: routes.py

- 30-second timeout on all ElevenLabs calls — prevents hung threads causing denial of service
- API keys passed in HTTP headers (xi-api-key) — never in URL query parameters that appear in server access logs

3.12 Frontend Security

File: index.html

- JWT stored to localStorage on login; cleared via localStorage.removeItem() on logout
- Session restored on page reload via IIFE reading token from localStorage on DOMContentLoaded
- showMsg() uses.textContent for user-influenced strings — prevents XSS via error message injection
- showMsgHTML() is a separate function used only for hardcoded internal HTML (loading spinners)
- Password fields use type='password' by default with aria-labelled visibility toggle
- CSP meta tag restricts resource loading to known safe origins

4. Controls Not Implemented — Technical Justification

The following security controls were identified, understood, and deliberately excluded. Each has a precise technical reason — none are implementation failures.

Control	Why Not Implemented
Multi-Factor Authentication	Requires registered domain for Supabase OTP redirect URLs — 127.0.0.1 is rejected as a callback target
Email Verification on Signup	Supabase verification emails contain redirect links to the registered Site URL — localhost has no callback destination
Password Reset Flow	Same root cause — reset tokens expire unused because they redirect to localhost
HTTPS / TLS Encryption	SSL certificates require a domain name; Let's Encrypt does not issue certificates for IP addresses
HttpOnly Secure Cookies	The Secure cookie flag is stripped by browsers over HTTP — localStorage is the pragmatic local alternative
HSTS	Browsers ignore Strict-Transport-Security unless delivered over HTTPS
OAuth / SSO	OAuth providers explicitly reject http://127.0.0.1 as a valid redirect URI for web applications

WebAuthn / Passkeys	Browser enforces that the rpId must be a registered domain — hard browser-enforced constraint
Production Rate Limiting	Application-level rate limiting resets on server restart; meaningful protection requires nginx/Cloudflare with persistent state
JWT Token Revocation	JWTs are stateless by design — revocation requires a Redis token blacklist not in current scope
Persistent Audit Logging	uvicorn stdout logs are ephemeral — production logging requires Datadog or CloudWatch